

IMPLEMENTAÇÃO DE UMA REDE NEURAL CONVOLUCIONAL PARA ESTIMATIVA DE AMPLITUDE DE SINAIS DO TILECAL NO PROCESSADOR SAPHO EM FPGA

Clethie S. B. Ngatsono[1] - bisseyou.ngatsono@estudante.ufjf.br
Chrysthofer A. A. Afonso[1] - chrysthofer.afonso@estudante.ufjf.br
Leandro G. Ribeiro[1] - ribeiro.leandro@estudante.ufjf.br
Diogo A. Cardinot[2] - diogo.cardinot@iprj.uerj.br
Luciano M. de Andrade Filho[1] - luciano.andrade@ufjf.br
[1]Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN),
Universidade Federal de Juiz de Fora – Juiz de Fora – MG
[2]Instituto Politécnico, Universidade do Estado do Rio de Janeiro – Nova Friburgo – RJ

Resumo. *A estimativa da amplitude de sinais do calorímetro de telhas (TileCal) do experimento ATLAS apoia-se em estimadores lineares, cujo desempenho se degrada sob empilhamento (pile-up). As redes neurais convolucionais (Convolutional Neural Networks, CNNs) aproximam essa relação de forma não linear, ao custo de muitas operações aritméticas. Este trabalho porta uma CNN unidimensional (duas camadas convolucionais e três densas, cerca de 11 mil pesos) para o processador soft-core SAPHO, cuja arquitetura é gerada sob demanda em FPGA, com o apoio da Aurora Intelligence, a assistente de IA da IDE AURORA, que propôs inclusive a consolidação da rede, antes repartida em dois processadores, em um único núcleo. O problema central é de fidelidade numérica: o modelo treinado em Python precisa ser reproduzido em uma aritmética de ponto flutuante própria, distinta do padrão IEEE 754. A saída do SAPHO reproduz a da referência com erro médio absoluto de* $8{,}17 \times 10^{-5}$ *e coeficiente de determinação* $R^2 = 0{,}9999997$ *sobre 20 pulsos reais, ocupando 296 instruções e 11.759 palavras de dados. Os resíduos são sistematicamente positivos, revelando um viés de arredondamento da aritmética própria, desprezível frente à resolução do detector. Evidencia-se a viabilidade de embarcar uma CNN complexa em um soft-core mínimo.*

Keywords: *Modelagem em hardware reconfigurável, Redes neurais convolucionais, Fidelidade numérica, Processador soft-core SAPHO, TileCal.*

1. INTRODUÇÃO

A física de altas energias investiga os constituintes fundamentais da matéria e as interações que os regem. Para sondar essa escala, a Organização Europeia para a Pesquisa Nuclear (em francês, *Conseil Européen pour la Recherche Nucléaire*, CERN) opera o maior complexo de aceleradores do mundo, cujo principal instrumento é o Grande Colisor de Hádrons (*Large Hadron Collider*, LHC), no qual feixes de prótons se cruzam a uma taxa de 40 MHz (Evans and Bryant, 2008). Entre os detectores que registram essas colisões, o ATLAS (*A Toroidal LHC ApparatuS*) destaca-se por seu amplo programa experimental (ATLAS Collaboration, 2008).

Boa parte de sua capacidade de medida provém dos calorímetros, que absorvem as partículas e medem a energia depositada; em particular, o calorímetro de telhas (*Tile Calorimeter*, TileCal) reconstrói a energia de jatos hadrônicos (jorros de partículas) a partir de pulsos elétricos digitalizados (Aad et al., 2010).

A energia depositada em cada célula do TileCal é proporcional à amplitude do pulso amostrado em uma janela de poucas amostras. O método clássico para estimar essa amplitude é a filtragem ótima (*Optimal Filtering*, OF), que combina linearmente as amostras com pesos que minimizam a variância do ruído (Cleland and Stern, 1994). Sob alta luminosidade (alta taxa de colisões), porém, sinais de colisões vizinhas se sobrepõem no tempo, fenômeno conhecido como empilhamento (*pile-up*), que introduz um viés dependente da ocupação e degrada a estimativa linear. A simulação fiel desses pulsos e a avaliação de estimadores em *hardware* foram tratadas recentemente no próprio ecossistema do grupo (Ribeiro, 2025).

Como alternativa aos estimadores lineares, métodos de aprendizado profundo (*deep learning*) têm se mostrado eficazes em aproximar funções não lineares diretamente dos dados (LeCun et al., 2015; Goodfellow et al., 2016). Entre eles, as redes neurais convolucionais (*Convolutional Neural Networks*, CNNs) extraem padrões locais por meio de filtros aplicados ao longo do sinal (LeCun et al., 1998), o que as torna candidatas naturais à estimação da amplitude a partir da forma do pulso, inclusive sob empilhamento. O preço dessa maior expressividade é computacional: uma CNN exige um grande número de operações aritméticas, o que precisa ser conciliado com a restrição de processamento em tempo real do experimento. Redes neurais já foram aplicadas à reconstrução de sinais do TileCal (Luna et al., 2024), mas essas avaliações ocorreram em software ou em aceleradores dedicados de grande porte em FPGA. O próprio grupo já embarcou uma rede perceptron de múltiplas camadas (*Multilayer Perceptron*, MLP) em processadores multicore em FPGA para a reconstrução *on-line* de energia no TileCal (Aguiar, 2023). Até onde se tem conhecimento, contudo, nenhuma CNN havia sido embarcada em um processador soft-core compacto e gerado sob demanda; é essa lacuna (a viabilidade de uma implementação mínima e customizável) que este trabalho aborda pela primeira vez.

O processamento *on-line* dos sinais do TileCal, em tempo real e com baixa latência, é realizado em *Field-Programmable Gate Arrays* (FPGAs), dispositivos reconfiguráveis que aliam paralelismo massivo e eficiência energética (Compton and Hauck, 2002). Quando o algoritmo a ser embarcado é extenso, recorre-se a processadores programáveis descritos em *hardware* (*soft-cores*); para evitar o desperdício de área lógica das arquiteturas fixas, o Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN)[1], da Universidade Federal de Juiz de Fora (UFJF), desenvolveu o SAPHO (*Scalable-Architecture Processor for Hardware Optimization*), um *soft-core* cuja arquitetura é gerada sob demanda, sintetizando em *hardware* apenas os recursos exigidos por cada programa (Kapisch et al., 2026), e já validado em aplicações de alta carga aritmética, como a geração de fractais em FPGA (Andrade Filho et al., 2025). O SAPHO é programado na linguagem de médio nível C$\pm$ (lê-se "C mais-menos"), derivada de C e estendida às necessidades do processador, e tem seu fluxo de projeto orquestrado pela AURORA (*Advanced Utility Running Optimized Resource Architectures*), a IDE (*Integrated Development Environment*) que integra edição, compilação, simulação e a assistente de inteligência artificial Aurora Intelligence (Chrysthofer Arthur et al., 2026).

Neste contexto, este trabalho descreve a implementação de uma CNN para a estimação da amplitude de pulsos do TileCal no processador SAPHO, conduzida com o auxílio da Aurora

[1]Em parceria com o CERN. Mais informações em `https://www.nipscern.com/projects/sapho`.

Intelligence. O foco não está na física da reconstrução, mas na integração entre a assistente de IA da AURORA e o SAPHO, que viabiliza levar uma rede convolucional complexa, com camadas convolucionais e densas e aritmética de ponto flutuante própria, a um único *soft-core* mínimo gerado sob medida. Mostra-se que a saída da rede embarcada reproduz, dentro de uma tolerância estreita, a do modelo de referência em Python. O restante do artigo está organizado da seguinte forma: a Seção 2 descreve, de modo sucinto, o processador SAPHO e a Aurora Intelligence; a Seção 3 detalha a arquitetura da CNN e a sua conversão para a linguagem C± do SAPHO; a Seção 4 apresenta os resultados, com a comparação dos valores estimados e as estatísticas de ocupação de *hardware* e de instruções; e a Seção 5 reúne as conclusões e os trabalhos futuros, com destaque para a execução em *hardware*.

2. O PROCESSADOR SAPHO E A AURORA

O SAPHO é um processador *soft-core* cuja arquitetura é gerada sob demanda: em vez de instanciar um núcleo de tamanho fixo, a ferramenta dimensiona as memórias de programa e de dados e os circuitos aritméticos conforme os requisitos de cada aplicação, sintetizando em *hardware* apenas o necessário. A organização segue o modelo Harvard, de memórias separadas de programa e de dados, com um *pipeline* de três estágios e uma unidade lógica e aritmética combinacional, configurável para operações em ponto fixo ou flutuante. Um traço decisivo para este trabalho é a sua aritmética de ponto flutuante própria, de 32 bits (um de sinal, oito de expoente e vinte e três de mantissa), que não segue o padrão IEEE 754 (a norma usual de ponto flutuante): é nela que a CNN, originalmente treinada com a aritmética do Python, precisa ser reproduzida com fidelidade. A Tabela 1 resume os parâmetros de configuração do SAPHO empregados.

Tabela 1: Parâmetros do processador SAPHO utilizados

Parâmetro	**Valor**	**Significado**
`#PRNAME`	`cnn_sapho`	Nome do processador/programa
`#NUBITS`	32	Largura do dado (float próprio, não IEEE 754)
`#NBMANT`	23	Bits de mantissa
`#NBEXPO`	8	Bits de expoente
`#NDSTAC`	64	Profundidade da pilha de dados
`#SDEPTH`	16	Profundidade da pilha de endereços
`#NUIOIN`	1	Porta de entrada (inteiro, escala 2^{20})
`#NUIOOU`	1	Porta de saída (inteiro, escala 2^{20})

Fonte: Autores (2026)

O desenvolvimento para o SAPHO é conduzido na IDE AURORA, que integra a edição em C±, a compilação pela cadeia YANC (*Yet Another Compiler*), a simulação e a assistente de inteligência artificial Aurora Intelligence. Especializada na linguagem e no fluxo do processador, ela atua como par de programação sob revisão dos autores: sugere código, diagnostica erros e propõe transformações coerentes com as regras do SAPHO. Esse conhecimento de domínio, que uma assistente genérica não teria, faz da portação um exercício do próprio ecossistema. Um exemplo concreto foi a consolidação da arquitetura: a rede, a princípio repartida em dois núcleos especializados, um para as camadas convolucionais e outro para as densas, foi reescrita em um único processador, o `cnn_sapho`, com o apoio da assistente. É essa integração entre a inteligência e o processador que torna viável levar uma

rede convolucional complexa a um *soft-core* mínimo, como se detalha na Seção 3.

3. ARQUITETURA DA CNN E CONVERSÃO PARA C±

Uma CNN leve é especialmente adequada à síntese em FPGA porque concentra a maior parte do seu custo computacional em multiplicações e adições sobre arrays de tamanho fixo, sem alocação dinâmica de memória, restrição fundamental em soft-cores. A não linearidade fica restrita à função de ativação: com ReLU, basta uma comparação e um multiplexador, mapeados pela primitiva pset() do SAPHO, ao contrário de funções como a sigmoide, que exigiriam a implementação de uma não linearidade bem mais custosa. O fluxo da AURORA IDE, com a Aurora Intelligence como assistente, acelera a portabilidade ao identificar automaticamente as restrições da linguagem C± e ao permitir compilar e simular partes da rede isoladamente antes de integrá-las ao processador final.

3.1 Arquitetura da CNN

A CNN adotada, proposta por Cardinot et al. (2026) para a reconstrução de sinais em calorimetria, recebe uma janela de sete amostras consecutivas do pulso digitalizado do TileCal, produzindo um único valor de ponto flutuante correspondente à amplitude estimada. A rede foi treinada com o otimizador Adam, função de perda MSE e 15 épocas. A Tabela 2 descreve as camadas em ordem de execução.

Tabela 2: Arquitetura da CNN 1D para estimativa de amplitude

Camada	**Tipo**	**Parâmetros**	**Saída**
1	Conv1D + ReLU + BatchNorm	32 filtros, kernel = 5	(7×32)
2	Conv1D + ReLU + BatchNorm	16 filtros, kernel = 3	(7×16)
3	Flatten	–	(112)
4	Dense + ReLU	112×64	(64)
5	Dense + ReLU	64×32	(32)
6	Dense (linear)	32×1	(1)

Fonte: Cardinot et al. (2026), adaptado pelos autores (2026)

Após o treinamento, os parâmetros da BatchNorm foram exportados na forma afim explícita $y = a \cdot x + b$ por canal, com coeficientes pré-calculados conforme a Eq. (1):

$$a_f = \frac{\gamma_f}{\sqrt{\sigma_f^2 + \varepsilon}}, \quad b_f = \beta_f - a_f \cdot \mu_f \tag{1}$$

onde γ_f, β_f, μ_f e σ_f^2 são o fator de escala, o deslocamento, a média e a variância aprendidos pela BatchNorm do filtro f, respectivamente. Todos os pesos foram exportados para arquivos `.txt` pelo script `export_sapho.py`, totalizando aproximadamente 11.185 valores de ponto flutuante.

3.2 Conversão para o C± e a Aurora Intelligence

A portação foi realizada em um único processador SAPHO (`cnn_sapho.cmm`, 102 linhas de código C±), com a fusão do BatchNorm, como transformação afim por canal ($a \cdot x + b$),

aritmética de ponto flutuante própria de 32 bits (1+8+23), entrada/saída inteira com escala 2^{20} e mapeamento de ReLU pela primitiva `pset()` do SAPHO. A Figura 1 resume esse fluxo de portação, do treinamento em Python à validação da rede embarcada.

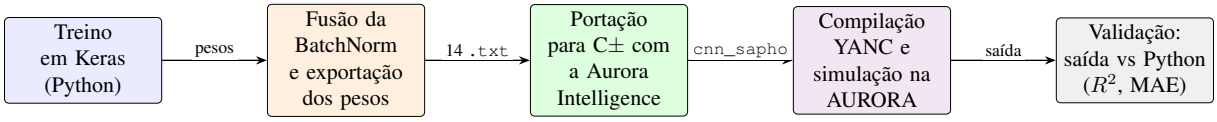


Figura 1: Fluxo de portação da CNN para o SAPHO: a rede treinada em Keras tem a *BatchNorm* fundida e os pesos exportados; a conversão para C±, apoiada pela Aurora Intelligence, gera o processador `cnn_sapho`, compilado e simulado na AURORA, cuja saída é validada contra a referência em Python.
Fonte: Autores (2026)

A Aurora Intelligence desempenhou papel central nessa portação: foi acionada diante de cada erro de compilação e sempre que uma nova restrição da linguagem se manifestava, como ilustra a Figura 2. Os pesos são carregados de arquivos externos usando a sintaxe nativa do C±, como em `float conv1_W[160] "conv1_W.txt";`, que vincula um *array* ao respectivo arquivo de valores. Esse mecanismo resolveu o principal obstáculo inicial: a impossibilidade de inicializar *arrays* globais extensos diretamente no código.

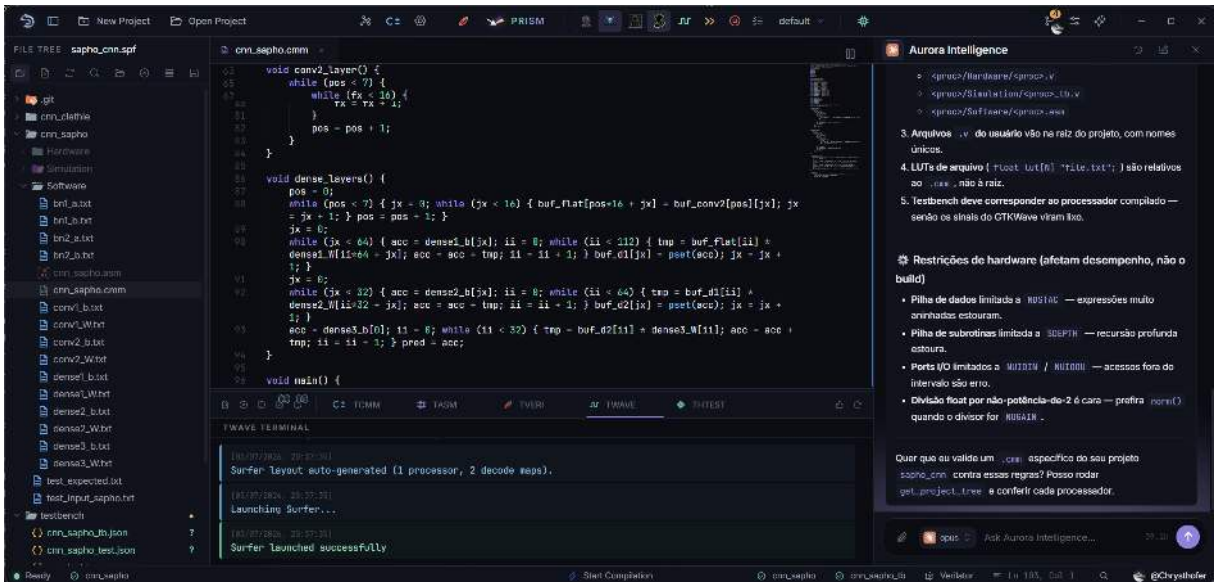


Figura 2: A IDE AURORA durante a portação da CNN. À esquerda, a árvore do projeto `sapho_cnn` expõe os pesos exportados como arquivos `.txt` (catorze ao todo, pesos e viés de cada camada parametrizada) ao lado do `cnn_sapho.cmm`; ao centro, o editor mostra a conversão dos laços `for` em `while` e o mapeamento da ReLU pela primitiva `pset()`; à direita, a Aurora Intelligence discute restrições de *hardware* do SAPHO (limites das pilhas `#NDSTAC`/`#SDEPTH`, portas `#NUIOIN`/`#NUIOOU` e o custo da divisão por não-potência de dois), complementares às restrições de *linguagem* sistematizadas na Tabela 3.
Fonte: Autores (2026)

3.3 Restrições do C± identificadas

A Aurora Intelligence auxiliou na identificação sistemática de sete restrições da linguagem C± ausentes na documentação oficial, listadas na Tabela 3.

Tabela 3: Restrições do C± e soluções adotadas

Restrição identificada	**Solução adotada**
Variável `i` é palavra reservada	Renomear para `ii`, `jx`, `kx`, `cx`, `pos`
Ausência do comando `for`	Substituir por `while` com incremento manual
Arrays não podem ser parâmetros de função	Promover todos os arrays a variáveis globais
Arrays globais sem inicialização `={}`	Carregar de arquivos `.txt` externos
Operador `>=` não interpretado corretamente pela IDE	Substituir expressões do tipo `x>=` por `x>-1`
Caracteres especiais em comentários	Usar apenas ASCII básico
Literais inteiros em contexto float	Substituir `0`, `1` por `0.0`, `1.0`

Fonte: Autores (2026)

4. RESULTADOS E DISCUSSÃO

4.1 Fidelidade numérica: SAPHO vs. modelo de referência

A validação foi conduzida comparando a saída do processador `cnn_sapho` com a do modelo de referência em Python para 20 amostras de teste com os pesos reais da CNN treinada. O conjunto tem esse tamanho porque o que se verifica é a equivalência numérica entre duas implementações do mesmo modelo, e não o desempenho estatístico da rede, já estabelecido por Cardinot et al. (2026): com pesos e topologia idênticos, qualquer divergência decorre da aritmética e se manifesta em toda amostra, não em uma fração delas. A Tabela 4 apresenta os resultados numéricos completos, sintetizados pelo erro absoluto médio (*Mean Absolute Error*, MAE) e pelo coeficiente de determinação (R^2), e a Figura 3 mostra os resíduos (erro do SAPHO em relação à referência Python) de cada amostra.

O $R^2 = 0{,}9999997$ e o MAE de $8{,}17 \times 10^{-5}$ confirmam que a conversão para C± preservou integralmente a precisão numérica da CNN sobre os 20 pulsos de teste.

Uma inspeção dos resíduos na Figura 3 revela um padrão sistemático, e não aleatório: os 20 erros são todos positivos, distribuídos entre $6{,}1$ e $10{,}9 \times 10^{-5}$, com baixa dispersão em torno da média (o RMSE de $8{,}25 \times 10^{-5}$ é praticamente igual ao MAE). Essa superestimação constante decorre do arredondamento nas conversões inteiro↔ponto flutuante (`I2F` e `F2I`) e da truncagem da mantissa de 23 bits do formato próprio do SAPHO (32 bits, 1+8+23), distinto do IEEE 754. Por ser aproximadamente constante e da ordem de 10^{-5}, o viés é desprezível frente à resolução física do TileCal e poderia, se necessário, ser absorvido em uma etapa de calibração, sem prejuízo à utilidade do estimador.

A Figura 4 ilustra, na simulação, o instante em que uma predição é emitida.

4.2 Ocupação de programa e dados

A compilação do processador `cnn_sapho` reportou 296 instruções Assembly e 11.759 palavras de dados, conforme indicado pelo TASM da AURORA IDE. Os ≈11.185 pesos

Tabela 4: Comparação entre o modelo de referência em Python e a saída do SAPHO

Amostra	**Ref. Python**	**Saída SAPHO**	**Erro abs.**
1	1.139896	1.139989	$9{,}3 \times 10^{-5}$
2	1.211246	1.211327	$8{,}1 \times 10^{-5}$
3	1.242645	1.242738	$9{,}3 \times 10^{-5}$
4	1.181293	1.181356	$6{,}3 \times 10^{-5}$
5	1.070004	1.070065	$6{,}1 \times 10^{-5}$
6	0.968662	0.968755	$9{,}3 \times 10^{-5}$
7	0.793167	0.793247	$8{,}0 \times 10^{-5}$
8	0.694679	0.694757	$7{,}8 \times 10^{-5}$
9	0.690926	0.691015	$8{,}9 \times 10^{-5}$
10	0.810413	0.810505	$9{,}2 \times 10^{-5}$
11	0.904121	0.904204	$8{,}3 \times 10^{-5}$
12	1.143490	1.143572	$8{,}2 \times 10^{-5}$
13	1.150639	1.150717	$7{,}8 \times 10^{-5}$
14	1.139347	1.139410	$6{,}3 \times 10^{-5}$
15	1.176285	1.176359	$7{,}4 \times 10^{-5}$
16	1.070190	1.070263	$7{,}3 \times 10^{-5}$
17	1.000095	1.000172	$7{,}7 \times 10^{-5}$
18	0.986122	0.986209	$8{,}7 \times 10^{-5}$
19	0.963333	0.963442	$1{,}09 \times 10^{-4}$
20	0.952862	0.952947	$8{,}5 \times 10^{-5}$
Erro absoluto médio (MAE)			$8{,}17 \times 10^{-5}$
Erro absoluto máximo			$1{,}09 \times 10^{-4}$
Erro relativo médio			$0{,}0084\%$
RMSE			$8{,}25 \times 10^{-5}$
R^2			$0{,}9999997$

Fonte: Autores (2026)

da CNN são carregados de arquivos `.txt` externos em tempo de inicialização, e não embutidos no código como cargas imediatas; é esse mecanismo nativo do SAPHO que mantém a memória de programa enxuta, com apenas 296 instruções, apesar do volume de parâmetros da rede.

As 11.759 palavras de dados abrigam esses pesos e os *buffers* intermediários das camadas. Como o SAPHO sintetiza em *hardware* apenas a largura de palavra, as profundidades de pilha e as portas de entrada e saída efetivamente declaradas (Tabela 1), o mesmo programa que exigiria superdimensionar um soft-core de tamanho fixo resulta, no SAPHO, em uma implementação compacta, compatível com FPGAs de porte moderado.

4.3 Latência de inferência

A simulação também permite medir a latência de forma exata, contando os ciclos de clock entre duas emissões consecutivas de `out_en`, o sinal que marca cada predição. Cada

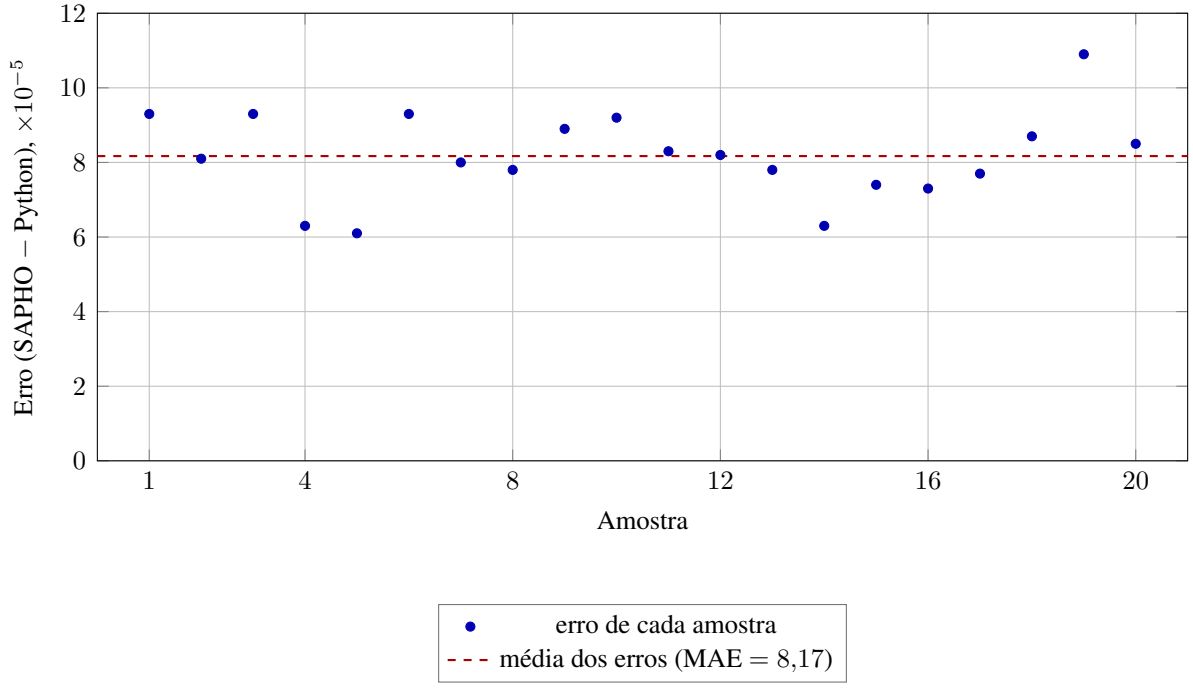


Figura 3: Resíduos da conversão para o SAPHO: erro (saída do SAPHO menos a do modelo de referência) de cada amostra, em unidades de 10^{-5}. Os 20 resíduos são todos positivos, revelando uma superestimação sistemática; a média (linha tracejada) coincide com o MAE de $8{,}17 \times 10^{-5}$ e o máximo é $1{,}09 \times 10^{-4}$.

Fonte: Autores (2026)

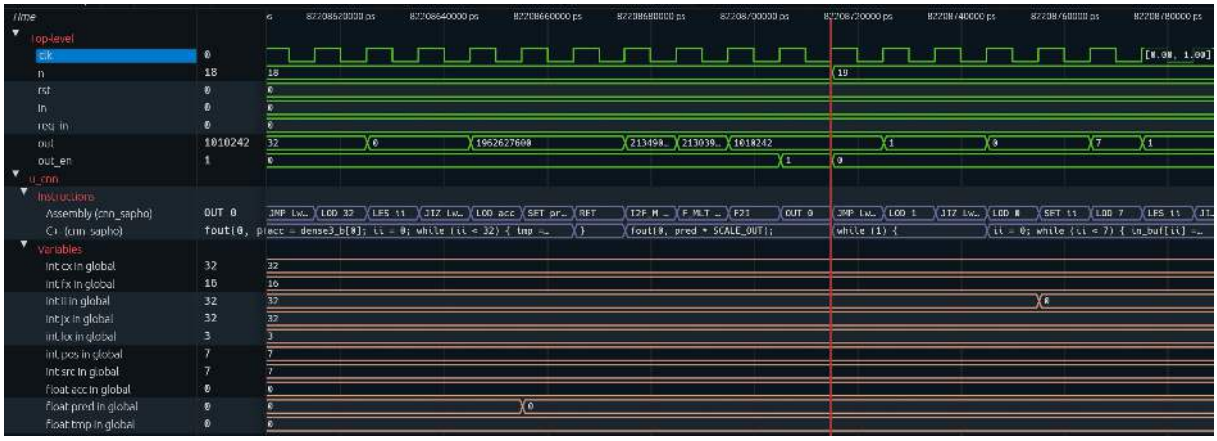


Figura 4: Emissão de uma predição pelo `cnn_sapho` no visualizador Surfer: ao final do processamento de uma amostra, o sinal `out_en` sobe e a porta de saída recebe `out` $=$ 1010242, isto é, a predição 0,9634 codificada como inteiro (escala 2^{20}). O *decode* automático exibe a instrução C± `fout(0, pred * SCALE_OUT)` e a sequência *Assembly* `I2F_M` $\rightarrow$ `F_MLT` $\rightarrow$ `F2I` $\rightarrow$ `OUT`.

Fonte: Autores (2026)

inferência consome 432.677 ciclos, valor idêntico para todas as 20 amostras: a latência é determinística, pois os laços da rede têm contagem fixa, sem desvios dependentes dos dados. À frequência de 100 MHz adotada na simulação, isso equivale a cerca de 4,33 ms por inferência (aproximadamente 231 inferências por segundo). Esses 100 MHz são um parâmetro do *testbench* para converter ciclos em tempo, e não uma frequência verificada nem um requisito do sistema de aquisição: a contagem de ciclos, essa sim, independe do valor adotado. Esse

custo reflete a natureza sequencial de um *soft-core*, que executa as ≈11 mil multiplicações e acumulações da rede uma a uma; a frequência máxima efetiva em *hardware* e os ganhos por paralelização dependem da síntese, tratada como trabalho futuro. A previsibilidade da latência, por outro lado, é uma propriedade desejável para sistemas de aquisição de dados em tempo real.

4.4 Limitações

A avaliação compara a saída embarcada com a do modelo de referência em Python, atestando a fidelidade numérica da portação, mas não avança sobre a acurácia física da CNN frente ao filtro ótimo em cenários de empilhamento, que permanece como trabalho futuro. A validação foi conduzida em simulação; a execução em *hardware* físico, com as métricas de síntese de área lógica e de frequência máxima, ainda não foi realizada. Além disso, o ponto flutuante próprio do SAPHO, não IEEE 754, introduz o viés sistemático observado (da ordem de 10^{-5}), que deve ser considerado em aplicações que exijam reprodução idêntica à aritmética de referência. Por fim, o conjunto de teste, de 20 pulsos, é limitado, e a portação, embora acelerada pela Aurora Intelligence, ainda demanda intervenção manual.

5. CONCLUSÕES E TRABALHOS FUTUROS

A integração entre a Aurora Intelligence e o SAPHO viabilizou portar uma CNN complexa, com duas camadas convolucionais e três densas e ≈11 mil pesos, a um *soft-core* mínimo, reproduzindo a saída do Python dentro de tolerância estreita (MAE $= 8{,}17 \times 10^{-5}$, $R^2 = 0{,}9999997$ sobre 20 pulsos reais). As principais contribuições são: (i) a identificação e documentação de sete restrições não documentadas da linguagem C±; (ii) a demonstração do uso da Aurora Intelligence como ferramenta eficaz na portação de algoritmos de aprendizado profundo para o SAPHO; e (iii) a validação numérica da CNN embarcada com erro inferior a 10^{-4}. O SAPHO e a IDE AURORA, além do uso em pesquisa, são adotados na cadeira de Dispositivos Lógicos Programáveis (DLP) da graduação em Engenharia Elétrica e em pesquisas de pós-graduação, de modo que portações como esta realimentam também a formação em *hardware* reconfigurável. Como trabalhos futuros, pretende-se realizar a execução em *hardware* físico, obtendo métricas de síntese no Quartus (LEs, registradores e frequência máxima), o que converterá os 432.677 ciclos por inferência em latência absoluta e abrirá caminho para a paralelização em *hardware*, além de avaliar a acurácia da CNN embarcada frente ao método clássico de filtragem ótima em cenários de empilhamento.

Uso de inteligência artificial

A Aurora Intelligence, assistente de inteligência artificial integrada à AURORA, foi utilizada como par de programação na portação da CNN para a linguagem C± do SAPHO, sugerindo código, diagnosticando erros e apoiando a consolidação da arquitetura em um único processador. Todo o código, os resultados e a redação foram revisados e validados pelos autores.

AGRADECIMENTOS

Os autores agradecem à Universidade Federal de Juiz de Fora (UFJF) e ao Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN) pela infraestrutura e pelo suporte contínuo, e ao CERN pela parceria científica que orienta as linhas de pesquisa do laboratório. Agradecem ainda o apoio do CNPq e da FAPEMIG. O presente trabalho foi realizado com apoio da CAPES, Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil, no âmbito do Programa CAPES/COFECUB. Declara-se, conforme a seção 2.8 destas normas, o uso da ferramenta de inteligência artificial generativa Claude (Anthropic) na revisão e na edição do texto deste trabalho.

Referências

Aad, G. et al. (2010). The ATLAS Tile calorimeter: Design, construction and performance. *The European Physical Journal C*, 70:1193–1236.

Aguiar, M. S. (2023). Implementação embarcada em FPGA de métodos visando a reconstrução online de energia no calorímetro hadrônico do experimento ATLAS. Dissertação (mestrado em engenharia elétrica), Universidade Federal de Juiz de Fora (UFJF), Juiz de Fora, MG, Brasil.

Andrade Filho, L. M., Chrysthofer Arthur, A. A., Kapisch, E. B., Castro, A. L. B. d., Rimes, S. d. M., and Libotte, G. B. (2025). Implementação de fractal em FPGA usando o processador soft-core SAPHO. In *Anais do XXVIII Encontro Nacional de Modelagem Computacional (ENMC) e XVI Encontro de Ciência e Tecnologia de Materiais (ECTM)*.

ATLAS Collaboration (2008). The ATLAS experiment at the CERN large hadron collider. *Journal of Instrumentation*, 3:S08003.

Cardinot, D. A., Peralva, B. S.-M., Libotte, G. B., and Andrade Filho, L. M. d. (2026). Convolutional neural networks for signal reconstruction in high-energy calorimetry. *Applied Sciences*, 16(13):6414.

Chrysthofer Arthur, A. A., Torres, G. A., Santos, P. H. S. R., and Andrade Filho, L. M. (2026). AURORA: Ambiente de desenvolvimento integrado para o processador soft-core SAPHO. Projeto de código aberto do NIPS-CERN, Universidade Federal de Juiz de Fora (UFJF). Disponível em: `https://www.nipscern.com/projects/aurora`.

Cleland, W. E. and Stern, E. G. (1994). Signal processing considerations for liquid ionization calorimeters in a high rate environment. *Nuclear Instruments and Methods in Physics Research Section A*, 338(2–3):467–497.

Compton, K. and Hauck, S. (2002). Reconfigurable computing: A survey of systems and software. *ACM Computing Surveys*, 34(2):171–210.

Evans, L. and Bryant, P. (2008). LHC machine. *Journal of Instrumentation*, 3:S08001.

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press, Cambridge, MA.

Kapisch, E. B., Aguiar, M. S., Andrade Filho, L. M., and Manso Silva, L. R. (2026). SAPHO—scalable-architecture processor for hardware optimization: An FPGA customizable implementation approach. *IEEE Access*, 14:6820–6832.

LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep learning. *Nature*, 521(7553):436–444.

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324.

Luna, F. C., Dias, U. F., Lisboa, P. H. B., Paschoalin, T. C. A., Quirino, T. M., and Andrade Filho, L. M. (2024). Real-time FPGA-based simulator for the Tile calorimeter readout system in the ATLAS experiment. In *Anais do XXVII Encontro Nacional de Modelagem Computacional (ENMC) e XV Encontro de Ciência e Tecnologia de Materiais (ECTM)*, Ilhéus, BA, Brasil.

Ribeiro, L. d. G. (2025). Simulador de pulsos do TileCal em tempo real utilizando o processador SAPHO e FPGA. Dissertação (mestrado em engenharia elétrica), Universidade Federal de Juiz de Fora (UFJF), Juiz de Fora, MG, Brasil.

APPENDIX A: ENGLISH TITLE, ABSTRACT AND KEYWORDS

IMPLEMENTATION OF A CONVOLUTIONAL NEURAL NETWORK FOR TILECAL SIGNAL AMPLITUDE ESTIMATION ON THE SAPHO SOFT-CORE PROCESSOR IN FPGA

Abstract. *Amplitude estimation of ATLAS Tile Calorimeter (TileCal) signals relies on linear estimators, whose performance degrades under pile-up. Convolutional Neural Networks (CNNs) approximate this relationship non-linearly, at the cost of many arithmetic operations. This work ports a one-dimensional CNN (two convolutional and three dense layers, about eleven thousand weights) to the SAPHO soft-core processor, whose architecture is generated on demand in FPGA, supported by Aurora Intelligence, the AI assistant of the AURORA IDE, which also proposed consolidating the network, initially split across two processors, into a single core. The core problem is one of numerical fidelity: the model trained in Python must be reproduced in a proprietary floating-point arithmetic that departs from the IEEE 754 standard. The SAPHO output reproduces the Python reference with a mean absolute error of* 8.17×10^{-5} *and a coefficient of determination* $R^2 = 0.9999997$ *over 20 real pulses, occupying 296 instructions and 11,759 data words. The residuals are systematically positive, revealing a rounding bias of the proprietary arithmetic that is negligible against the detector's resolution. The results show the feasibility of embedding a complex CNN in a minimal soft-core.*

Keywords: *Modelling on reconfigurable hardware, Convolutional neural networks, Numerical fidelity, SAPHO soft-core processor, TileCal.*